THE REVIEW

Bitcoin Machine Code, 2026-10-23: a security review, a formal-verification pass, and every fix

The fifth report in the series that began with "21 FOR 21"

Bitcoin Machine Code · github.com/BobClawblaw/bitcoinmachinecode

A one-day report. On 2026-10-23 the node was given a security and code review of everything written since the 09-03 audit closed, every finding was fixed, and then four areas of the code were put through formal verification. The proofs found defects the review had not, and fixing those turned up two more that had been live for weeks. This report covers all of it: what was found, how each finding was confirmed, what changed, how each fix was tested, and how the final build was checked against Bitcoin Core v31.1 on the real chain. It was compiled from the project's own records: the commit messages of the twelve pull requests (#424 to #435), the audit report (docs/audits/SECURITY_AUDIT_2026-10-23.md), the formal-verification READMEs and run logs, and the validation-sync harness logs. Times are UTC.

> "A bounded check is not a proof. Every result must state its bound and its trusted base."
> *-- the formal-verification rules, 2026-10-23*

The short version:

- **The review** found 9 HIGH, 18 MEDIUM and 36 LOW findings, plus a few INFO items. Five of the HIGHs were consensus splits from Core: rules where bmc accepted, or refused, a block that Core would not. One let any peer get a real block marked invalid without doing any proof of work.
- **Everything was fixed** in seven batches, each fix with a test watched to fail when the fix is reverted. A from-genesis mainnet sync of that build matched Core v31.1's UTXO set exactly at height 972,783.
- **Formal verification** of four areas (consensus arithmetic, the secp256k1 assembly, the UTXO store's crash consistency, and the parsers of untrusted input) found one proof-of-work encoding defect, six crash-consistency holes in the UTXO store (one critical), and seven parser divergences from Core.
- **Fixing those** turned up a C comma expression that had silently disabled two boot-time safety checks since 2026-09-05, and a set of `getblocks` defects, one of which made every `getblocks` reply name blocks nobody has.
- **The final build** passes the full gate (474 tests) and a second from-genesis sync: **MuHash identical to Core v31.1 at height 972,848**.
- **Landing:** twelve stacked pull requests, #424 to #435. Merged in order, they produce exactly the tested tree.

Findings, review	9 HIGH, 18 MEDIUM, 36 LOW, plus INFO
Findings, formal pass	1 PoW encoding (F1), 6 storage (STO-A/B/C/D/E/W), 7 parser (F1-F7)
Found while fixing	the dead UTX-2/UTX-6 checks (comma expression), 7 getblocks defects, 4 missing Makefile prerequisites
Commits	70 (non-merge), 179 files, +12,138 / -1,153 lines
Tests	474 in the gate (from 472 after the first seven batches); every fix revert-checked
Validation sync #1	seven batches: PASS at 972,783, MuHash identical to Core
Validation sync #2	final build: PASS at 972,848, MuHash fe5ab3ec...559a90a identical to Core
Pull requests	#424 to #435, stacked, all mergeable

Contents

PROLOGUE: WHAT WAS ASKED, AND HOW IT WAS DONE 5

PART I: THE SECURITY REVIEW 6

Chapter 1: Scope and Summary 6

Chapter 2: The HIGH Findings 6

Chapter 3: The MEDIUM and LOW Findings 7

Chapter 4: What Held 8

PART II: THE SEVEN BATCHES 9

Chapter 5: The Fixes 9

Chapter 6: How Each Fix Was Checked 10

Chapter 7: Validation Sync #1, and the 06:47 Incident 10

PART III: FORMAL VERIFICATION 12

Chapter 8: The Rules 12

Chapter 9: Consensus Arithmetic 12

Chapter 10: secp256k1 13

Chapter 11: Storage Crash Consistency 13

Chapter 12: The Parsers 14

PART IV: WHAT THE PROOFS LED TO 16

Chapter 13: The Store's Durability Order (#431, #432) 16

Chapter 14: The Comma Expression (#433) 16

Chapter 15: The Parser Fixes (#434) 17

Chapter 16: getblocks, Read Again (#435) 17

PART V: THE FINAL BUILD 19

Chapter 17: The Gate 19

Chapter 18: Validation Sync #2 19

Chapter 19: Landing 19

APPENDIX A: WHAT IS STILL OPEN 21

APPENDIX B: BEHAVIOUR CHANGES FOR OPERATORS 22

PROLOGUE: WHAT WAS ASKED, AND HOW IT WAS DONE

The day began with a request for "another security and code review of the bmc codebase" and a vulnerability assessment. The last full audit had closed on 2026-09-03. Since then about 1,060 commits had landed: the parallel applier, the coinstats history index, compact-block receive, the B15 and B16 storage reads, a new mempool policy engine, and much else. The review was weighted toward that new code.

The method was the one the project has used since August, with one rule learned the hard way. The code was split into six areas, reviewed in parallel. Each finding then had to be confirmed against the code, because earlier audits had shown that review findings are not trustworthy as written: some were already fixed, some understated, and some came with a suggested fix that would itself have introduced a bug. The audit report marks each finding V (re-read end to end against the code and against Core v31.1's source), A (traced by the area reviewer) or P (plausible, one named link unverified).

The fixing rule was older still. A test is not trusted until the fix is reverted and the test is watched to fail. Six tests had once passed against unfixed code, one of them a fixture written after a fork. Every fix in this report has such a test.

After the fixes, the request was for "a formal verification of bmc", with all four proposed scopes: the consensus arithmetic, the secp256k1 assembly, the storage layer's crash consistency, and the C parsers of untrusted input. The tools were installed in a user-local directory: CBMC 6.11, z3 5.3.1, TLA+ (TLC 2.19 on OpenJDK 21), plus a small bit-precise x86-64 symbolic executor written for the secp256k1 proofs. Each area has a `run.sh` that exits 0 only if every proof holds and every mutant is caught.

PART I: THE SECURITY REVIEW

Chapter 1: Scope and Summary

The review read `asm/` at f07b90bb. Nothing was built or run against a node; denial-of-service costs in the findings are timings of copied code or estimates. The full report, with file and line references for each finding, is docs/audits/SECURITY_AUDIT_2026-10-23.md.

Sev	Count	Areas
HIGH	9	consensus (CON-1..5), p2p (P2P-1), mempool (MPL-1..3)
MEDIUM	18	rpc (RPC-21..23), consensus (CON-6..9), mempool (MPL-4..6), config (CFG-1, CFG-2), wallet (CRY-1), storage (ST2-1..4), p2p (P2P-2)
LOW	36	crypto (CRY-2..8), p2p (P2P-3, P2P-4), rpc (RPC-24..27), consensus (CON-10..14), mempool (MPL-7, MPL-8), config (CFG-3..6), host (HST-1..7), storage (ST2-5..9)

No CRITICAL memory-safety finding was found on the remote surfaces. The new constant-time point code, the field and scalar kernels, safegcd, the MuHash inversion, ChaCha20-AVX2, Poly1305 and ElligatorSwift all matched their references on edge-case differentials.

Chapter 2: The HIGH Findings

CON-1: any peer could get a real block marked invalid. The connect path ran the block weight check before the witness-commitment check, and a weight failure was filed as a real refusal. A coinbase whose witness walk failed was filed as `bad-witness-merkle-match`, also a real refusal. A real refusal invalidates the block's hash: it is written to invalid.dat, the archive is truncated and the headers roll back. Blocks are archived after only `cons_verify` (PoW, txids, merkle root), none of which covers witness bytes. So a peer could take a real block, add junk witness items to push its weight over 4M (or empty the coinbase's witness stack, or change the nonce length), and serve it. Core v31.1 runs `CheckWitnessMalleation` before `bad-blk-weight` and files all of these as BLOCK_MUTATED, which never marks a block failed.

P2P-1: the keep-up pass stored the wrong block. `node_sync_multi` computed each requested hash from the peer's own `headers` reply and filed whatever `block` came back under it. Nothing compared the body's hash with the requested one; `idxscan_append_locked` checked only that the body extended the tip. `cons_verify` checks a block's PoW against its own nBits, so a block at the easiest target (about 2^32 hashes) passed. The applier then saw index hash X over body Y and stopped at that height, and because index.dat is durable the stop survived restarts.

CON-2: unknown witness versions skipped EVAL_FALSE. Core's `VerifyScript` evaluates the scriptPubKey first and fails EVAL_FALSE on a false top item, before the witness branch. bmc classified any unknown witness version as an automatic pass. An output `52 02 0000` (OP_2, then a push of 0x0000, which is false) was spendable with an empty scriptSig on bmc and not on Core.

CON-3: zero-output transactions were accepted on connect. Only zero inputs were refused. Core refuses `bad-txns-vout-empty`.

CON-4: a superfluous witness on non-coinbase transactions. With a witness commitment present, only the coinbase was walked for witness structure; the other transactions were hashed for wtxids. A `00 01` marker with no witness data on any other transaction was accepted. Core throws "Superfluous witness record" at deserialization.

CON-5: nVersion was compared unsigned. Core's `nVersion` is `int32_t`, so any version at or above 0x80000000 is negative and fails `bad-version`. bmc read it as `unsigned int` and accepted it.

MPL-1: free repeated signature verification. A `tx` pushed unasked skipped the recent-rejects filter, which was consulted only before sending a `getdata`, and the P2P accept path ran scripts before fees and policy. The RPC paths had been moved to Core's order on 10-03; this one had not. The attack: push the same large transaction with one bad signature again and again.

MPL-2: 1p1c re-verification amplification. Each arrival of a fee-rejected parent ran a full test on every parked orphan child, and a failing child stayed parked for the orphan TTL.

MPL-3: eviction re-linearized the whole pool. `worst_chunk_excl` rebuilt the child-edge index, found every cluster and linearized each one, on every call, under the pool lock. It ran once per eviction and once per replacement. At up to about 490 µs per adversarial 64-member cluster, one eviction could take seconds.

Chapter 3: The MEDIUM and LOW Findings

The MEDIUMs were bounded-resource and correctness defects:

- **RPC-21:** detached per-connection Esplora threads leaked about 24 MB of `__thread` buffers each, an unauthenticated memory exhaustion.
- **RPC-22:** the Esplora listener had no total read deadline, so 16 slow sockets locked out every client.
- **RPC-23:** unauthenticated routes did unbounded work, and responses carried `Access-Control-Allow-Origin: *`.
- **CFG-1:** RPC credentials were read section-blind: a `[regtest] rpcpassword=` applied on mainnet, and `rpcport` went through `atoi`. An earlier finding (DMN-4) had been marked closed, but this reader was never changed.
- **CFG-2:** an `rpcwhitelist` whose entries were all rejected failed open, to unrestricted.

- **CRY-1:** the legacy BMCWAL v2 wallet cipher turned out to be a public pad XOR one 64-byte key: 64 bytes of known plaintext decrypt the whole file. v2 has been read-only since 2026-09-02, and opening one rewrites it as v3, but backups and snapshots of v2 files are still exposed.
- **ST2-1:** the LSM assembly ignored every fsync result before publishing a manifest or unlinking compaction inputs.
- **ST2-4:** the undo store was never fsynced, so a power loss could make WAL operations durable without their undo.
- **The rest:** CON-6..9 (a fixed 4 MiB taproot sighash cap, assumevalid by height rather than ancestry, the missing BIP94 timewarp rule on testnet4, and a non-canonical tx-count that halted the applier); MPL-4..6 (whole-pool scans in admission and replacement, and one peer able to flush the global orphan pool); P2P-2 (a full mempool scan for an unsolicited compact block before any PoW check); ST2-2 and ST2-3 (overlapping addr_hist runs, and `dumptxoutset` overwriting any file in the chain directory).

The LOWs ranged across the code:

- **Crypto:** a BIP340 borrow bug (CRY-2) that made signing report success with an invalid signature for some keys.
- **Wallet:** secret wipes the compiler had removed.
- **Host:** side files created 0644 and then chmodded; a Tor control client that sent the plain cookie.
- **Config:** grammar differences from Core's bitcoin.conf.
- **Consensus caps Core does not have:** a 100-block reorg limit, a 64-entry invalid set, and a submitblock transaction cap below Core's.

Chapter 4: What Held

The review also recorded what it checked and found sound:

- **v1 framer:** the 4 MB cap before draining, and checksum verification.
- **BIP324:** the garbage bound, decoys, handshake deadlines, and the constant-time tag compare.
- **Compact-block receive:** the commitment check and the collision fallback.
- **Addr:** caps and scoring.
- **RPC auth:** the constant-time compare and the cookie mode.
- **Consensus:** merkle and CVE-2012-2459, MAX_MONEY, CVE-2018-17144 duplicates, maturity, BIP30/34, BIP68 with no input cap, retargeting, the interpreter limits.
- **Crypto:** the CT table scans, and RNG fail-closed.
- **Storage:** the pread path, the append-cursor invariant, and the manifest's tmp, fsync, rename and directory-fsync sequence.
- **Build:** NX, full RELRO, FORTIFY and stack protector on the daemon.

PART II: THE SEVEN BATCHES

Chapter 5: The Fixes

The findings were fixed on seven branches, one per area, each starting from the same main (fc5e56a2). Each is now one pull request.

PR	Batch	Commits	Findings
#424	consensus	11	CON-1..14, and F1 from the formal pass
#425	p2p	3	P2P-1..4
#426	storage	9	ST2-1..9
#427	crypto and wallet	10	CRY-1..8, HST-7, INFO items, the Makefile prerequisites
#428	rpc	6	RPC-21..27
#429	config and host	8	CFG-1..6, HST-1..6, an I2P INFO item
#430	mempool and relay	2	MPL-1..8

Some of the changes worth describing:

- **CON-1:** witness-dependent failures are MUTATED and never invalidate the hash, and `bad-blk-weight` comes after the commitment check, in Core's order.
- **P2P-1:** the pass hashes the received header and requires the hash it asked for. Before `cons_verify` it runs Core's header acceptance (PoW against the header's own bits, then the contextual rules NET-5 already applied to inbound pushes). A header Core calls BLOCK_INVALID_HEADER scores the peer and closes the leg. `cmpct_recv_blocktxn` now takes the wanted hash, so it cannot complete another block's reconstruction. The applier drops a body that is not the block its index records after three passes, instead of retrying forever.
- **MPL-1:** a received transaction whose wtxid is in the reject filter is dropped, asked for or not, and refusals are filed by wtxid. Filing by txid would let a mangled witness shut out the valid transaction. Prechecks and policy now run before the scripts. No peer is scored for an invalid transaction: Core v31.1 does not score them either, so the audit's suggestion to add that was not taken.
- **A latent sigop pricing bug, found on the way:** `mpol_add_core` reads and clears the parked sigop cost, so running the policy test before the commit priced the entry at its plain vsize. That had been true of `sendrawtransaction` since 10-03: a 100-sigop, 170-byte transaction entered at 170 vB instead of Core's 2,000. It is fixed.
- **MPL-3:** each node now carries its cluster's last chunk, and only clusters that changed are re-linearized. On an unchanged pool of 400 clusters plus

3,000 singletons, one call costs 1 linearization and 0.10 ms, where the old scan cost 3,400 linearizations and 38 ms.

- **CON-10:** the reorg depth cap is gone. Undo data is kept for every block, and the reorg's pre-flight proves every height it will disconnect is readable before anything changes.
- **CON-14:** the `cons_verify` callers passed the scratch size in bytes where the function counts 32-byte slots, which is 32 times the real room. Separately, submitblock's transaction bound is now exactly Core's `bad-blk-length` boundary, (1,000,000 - 83) / 60 = 16,665.

Chapter 6: How Each Fix Was Checked

For each fix:

1. Write a test that fails on the unfixed code.
2. Apply the fix, and see the test pass.
3. Revert the fix: rebuild, check the binary's mtime so a stale binary cannot pass, and restore the file by copy rather than `git checkout`.
4. Watch the test fail.

The commit messages record each revert result. Some tests had encoded the old bugs, and were found and inverted rather than "fixed". One addr test asserted penalties that Core never applies. An fsync-count test pinned the old order. Peer-facing changes were also tested on the valid cases: canonical counts, a full 1,000-entry addr, valid BIP324 message types.

The seven branches were merged into an integration branch in order. The Makefile's test-list conflicts were resolved by union, and one real conflict, in `serve_hdrctx.c`, was resolved by moving the timewarp check into the shared `ctx_check`. The full gate on the result passed: 472 tests, `make` exit 0, and the gate-log audit (every gated test ran, no failure markers).

Chapter 7: Validation Sync #1, and the 06:47 Incident

Consensus changes are not landed on unit tests alone. At 06:05 a from-genesis mainnet sync of the integration build started. Its harness compares the UTXO set with an unpolled Bitcoin Core v31.1 at the same height once the node is ready, with networking off and the height held still.

At 06:47 a CBMC run in the formal-verification work, started without a memory cap, grew to 52 GB and triggered a global out-of-memory kill. The killer took the whole control group the run belonged to. The sync daemon, in the same group, shut down cleanly at height 503,361, and the harness monitor died with it. The sync was relaunched at 06:48 on the same datadir as its own service unit. From then on every solver ran under an 8 GB cap with swap off, one check at a time, and every long run was its own unit.

The node was ready at 13:22. The capstone was run by hand, since the monitor was gone: networking off, height stable for 60 s. bmc's MuHash, its per-height coinstats row and Core's row were identical at 972,783, and so were the block hashes. PASS.

PART III: FORMAL VERIFICATION

Chapter 8: The Rules

The pass ran under a written set of rules:

1. **Real code only.** Every proof runs on the real code: either text cut from the product file on every run (with `#line` back to it, and the run stops if an anchor is missing or matches twice), or the assembled objects themselves. Nothing is a hand-copied model of the code, except the TLA+ model, which cites file and line for every step.
2. **Every property can fail.** Each one is shown able to fail, by a mutant of the product code that it must catch, and checked for vacuity, with a cover goal it must reach.
3. **Bounds are stated.** A bounded result is reported as bounded, with its bound. A result that did not close is reported OPEN.
4. **Findings are confirmed on compiled code.** Every candidate is reproduced on the real compiled code before it is called a finding. Findings are reported, not fixed, by the verifier.

Chapter 9: Consensus Arithmetic

The arithmetic area set out to show that bmc's consensus arithmetic accepts exactly what Core v31.1 accepts, bit for bit, for all inputs. It covered:

- money (MoneyRange, output sums, input checks, fees, subsidy, coinbase maturity);
- lock times (median time past, IsFinalTx, BIP68, the header time and version floors);
- proof of work (SetCompact/GetCompact, CheckProofOfWork, the retarget, chainwork, BIP94);
- limits (size, weight, sigop cost, CompactSize, nVersion);
- script numbers and the arithmetic opcodes.

Tools: CBMC and z3.

Money, lock times and limits were proved or bounded-OK. Script numbers and the arithmetic opcodes were tested against Core but not proved. The end-to-end retarget harness did not close; its parts did. The final `run.sh` on the fixed code: **142 PASS, 0 FAIL, 1,589 s**.

F1, the one finding: `powr_compact` shifted a small target twice. For a target below 2^16 the size byte is under 3, and the mantissa must be shifted left once. bmc shifted it a second time. Retargeting the base `0x03000100` to a quarter of its target gave `0x01000000`; Core gives `0x01400000`. It is latent: no mainnet or test-network target is that small, because it would need a block

hash of about 2^18 or less. But it was wrong code in a consensus function. The fix (e0476262, in #424) drops the second shift. The CBMC property P3 now proves the encoding equal to Core's for every 256-bit target, and a mutant that restores the shift is caught.

Chapter 10: secp256k1

The secp256k1 field (mod p) and scalar (mod n) arithmetic was proved on the assembled objects, linked into an image and decoded by objdump. A bit-precise x86-64 symbolic executor, with an algebraic shadow of the carry chains in the style of CryptoLine, gave unbounded results. Products are never bit-blasted: a 64x64 multiply gives an abstract (hi, lo) pair with `hi * 2^64 + lo = x * y`, which is sound because the real product satisfies it.

The results cover:

- the field and scalar operations;
- the GLV split;
- `scalar_small_nonzero`, which refuses exactly k = 0 and k >= n;
- 26 constant-time properties.

80 of 80 proofs, 24 of 24 mutants caught, 1,416 s. There were no arithmetic findings.

Not covered: the safegcd inverse (differential tests only), batch inversion, wNAF, and the group-law formulas.

One observation from this area became a fix: `asm/Makefile` had no dependency on the `%included` `secp256k1_fe_inline.inc`, so editing it left a stale object that still linked. A search found three more such files (`g_comb_table.inc`, `wordlist.inc`, `script_flags_consts.inc`). All four are now prerequisites (cac2279d, in #427), checked with `make -n`.

Chapter 11: Storage Crash Consistency

The question: after a crash (SIGKILL or power loss) at any point, then recovery, is the UTXO set exactly the set after the recovered height? Is that height between the last durable checkpoint and the last applied block, and do the undo data and the archive index agree with it? Is anything durable lost or resurrected? And is recovery itself crash-safe?

The protocol was written out step by step with file and line citations, then modelled in TLA+. In the model, block b spends coin b-1 and creates coin b, so a resurrected spend shows up as an extra coin and a lost creation as a missing one. Every file has a page-cache state (what the next process sees after SIGKILL) and a durable state (what a power loss keeps). There are two file-system models: ext4 in ordered mode, and a weaker POSIX-only one. The largest configuration has 81 million states; the whole run takes 617 s.

TLC found five classes of hole. Four were reproduced on the real code by a crash-injection driver linked against the real objects:

ID	Sev	Defect	Consequence, reproduced
STO-E	critical	`fz_retire_wal` made utxo.idx's log offset durable with no WAL fsync	after a power loss new records land below the offset and are skipped on reload: a silently wrong UTXO set
STO-B	high	rollback buffered its WAL records while the undo slot was cleared at once	a kill during boot recovery: `bad-txns-BIP30`, and a valid block invalidated
STO-A	high	spends could become durable before their undo data	the same BIP30 failure on recovery
STO-C	medium	the block-index record was never synced	the boot refuses: applied height ahead of the archive tip
STO-D, STO-W	weak FS	directory-fsync results ignored; run files and manifest renames durable out of order	only on a file system that does not order a rename after its data

With five candidate fixes switched on in the model, every property holds under both file-system models. For STO-D and STO-W, the model also fails again with either fix removed.

Chapter 12: The Parsers

CBMC was run on the C code that parses bytes from the network or from disk, with every check flag on: bounds, pointers, signed and unsigned overflow, conversions, leaks, unwinding assertions. The checks were memory safety and the absence of undefined behaviour, plus postconditions and Core-equivalence. 14 functions (17 entries) were proved or bounded-OK, and **no memory-safety defect was found in any parser whose proof closed.** These did not close within the 8 GB cap and are OPEN:

- the BIP324 transport;
- compact-block receive;
- JSON-RPC;
- REST getutxos;
- RPC auth;
- ASMap;
- the undo readers;
- the fork tree;
- the config line splitter;
- the block witness commitment.

Seven divergences from Core were found and confirmed by concrete reproductions:

#	Was	Reach
F1	`getheaders` with a non-canonical locator count was answered	peers
F2	addr/addrv2 payloads Core drops were partly accepted (an assembly count reader read 4 of 8 bytes; truncated lists; wrong-length IPv4), and some malformed ones were scored at 100	peers
F3	BIP324 message types with control or high bytes were delivered	peers
F4	txindex run-file header checks could wrap, so a crafted file made a lookup read far past the mapping	datadir
F5	a mempool-journal header with sequence 0 made `bmcgetmempooljournal` walk 2^64 slots	datadir
F6	mempool.dat's delta-map count check could wrap	datadir
F7	a manifest generation or run number of 2^64-1 stopped the counter advancing	datadir

PART IV: WHAT THE PROOFS LED TO

Chapter 13: The Store's Durability Order (#431, #432)

STO-A, B, C and E have one fix, in the order Core's `FlushStateToDisk` uses. Before any UTXO byte reaches disk, the block files are synced, then the index, then undo. A durable-prefix hook, `live_durable_prefix`, runs ahead of every WAL drain. The WAL buffer grows to 32 MB, so that one sync covers many blocks.

The new test `test_sto_crash` drives the real code through kills and simulated power losses with a durable-image model of the files. Reverting the hook fails its A1, A2 and A3 cases.

The cost was measured with a new bench, `bench_utxo_apply_sync`:

- **in bulk:** 34 syncs per 1,000 blocks (was 13), with bulk sync time within noise;
- **at the tip:** about 2 extra syncs per block, 6-8% slower in the benchmark.

STO-D and STO-W (#432):

- **STO-D:** a failed directory sync now keeps the old state and retries. That costs redone work, never lost data.
- **STO-W:** one extra directory sync per manifest publish orders the run files before the rename.

`test_lsm_fsync_fail` now expects four fsync sites per flush and compaction. It had pinned the old order. Reverting the sweep fsync fails case W4.

Chapter 14: The Comma Expression (#433)

While adding one more boot check (refuse a manifest that names a missing run file), the reload's result turned out never to reach the checks that should have used it. `utxo_live_init` read it like this:

```
r = (g_utxo_in_reload = 1, utxo_lsm_reload(...), g_utxo_in_reload = 0)
```

In C, the value of a comma expression is its last operand, so `r` was always 0, whatever the reload returned. Two safety checks had been dead since TXOQ-1 (901475c7, 2026-09-05):

- **UTX-2:** a WAL tail bigger than the memtable. In the test, the node booted on a set truncated at 65,536 of 80,000 coins.
- **UTX-6:** an unreadable manifest. The node booted on an empty run set, and a later sweep deleted every real run.

There is no sign either ever fired in production. The production store's manifest names five runs and all five are present, and both validation syncs matched Core. But the protection had been missing for seven weeks.

The fix is three plain statements. A store that exists but will not reload now **halts**: nothing is applied, and the public and RPC tip stay at the last connected block. Before, it fell into the untracked mode, which advertises unvalidated blocks. A manifest that names a missing run halts too. Each case (STO-M, UTX-6 with a bad manifest, UTX-2 with 80,000 coins against a 2^16 table) was watched to fail with its fix reverted.

Chapter 15: The Parser Fixes (#434)

- **F1:** a non-canonical locator count drops `getheaders`, as in Core, and `getblocks` now parses its locator the same way.
- **F2:** nothing is taken from an addr payload Core would not deserialize whole, and only a well-formed vector over 1,000 entries is scored, as in Core. The test that had asserted Core-less penalties was inverted.
- **F3:** a BIP324 message type Core refuses is dropped, and the session continues.
- **F4 to F7:** the checks are overflow-safe. A journal header with sequence 0 disables the journal; mempool.dat's count is read as Core reads it; a manifest counter of 2^64-1 makes the manifest invalid, so the boot halts.

F1 to F3 are reachable from peers. F4 to F7 need a damaged or tampered file in the datadir.

Chapter 16: getblocks, Read Again (#435)

Work on F1's `getblocks` locator parsing turned up problems in the `getblocks` reply itself:

- **Byte order:** each block in the `inv` was named by its byte-reversed, display-order hash, so every reply announced blocks nobody has. The tip announce had had the same bug until 09-19.
- **hashStop** was ignored. It is now exclusive, as in Core.
- **Empty replies:** a `getblocks` with nothing to announce got an empty `inv`, and now gets none.
- **Continuation:** Core follows a `getdata` for the 500th announced block with `inv(tip)`, so the peer asks again. That is now implemented.
- **Locator limits:** a locator over 101 entries now disconnects, for both `getblocks` and `getheaders`. A locator with no known entry is served from height 1. An empty `getheaders` locator returns only the `hashStop` header.

A 10-second stall in the test harness after each `getheaders` was traced to a write to fd -1 that waited out a 10 s `poll()`. Production never writes to a

negative fd, but `fd_write_all` now fails such a write at once. The new `test_getblocks` fails with the byte order reverted.

PART V: THE FINAL BUILD

Chapter 17: The Gate

Every follow-up was merged into the integration branch after its own gate. The final integration (267308b4) passed the full gate: **474 gated tests ran, `make` exit 0, gate-log audit clean.** In all: 70 non-merge commits over main, 179 files, +12,138 and -1,153 lines.

Chapter 18: Validation Sync #2

A second from-genesis mainnet sync of the final build ran from 20:52 to 23:44, with parity checking against Core v31.1.

Source	integration branch at 267308b4 (the tree all twelve PRs land on)
Blocks stored	972,848 of 972,848, 0 bad, no download gaps
IBD end	10,007 s after launch
All indexes ready	10,024 s (UTXO, txindex, block filters, coinstats history)
Peak memory	PSS 61,091 MB, RSS 62,025 MB
Capstone	network off, quiesced at 972,848; Core asked for the same height
Result	**PASS: MuHash fe5ab3ec0d213162cd56e5149814a1fe3565cde75966968cd320d41f1559a90a identical to Core's, live answer agrees**

The node then shut down cleanly.

Chapter 19: Landing

The seven batches cannot land one at a time from the merge button. Every batch after the first conflicts with main in the Makefile's test list, and P2P also in `serve_hdrctx.c`. So the twelve branches were stacked in landing order, each with one merge of the branch below it:

- the seven batches take the integration branch's own conflict resolutions;
- the crypto batch's later Makefile fix is re-applied on top;
- each step was compared with the matching integration point, and differs only in the files of commits that came later.

The five follow-up branches were cut from the integration branch. Each lands on exactly the integration tree at its merge point.

Merging all twelve into main in order, with merge commits, was simulated. Every merge is clean, and **the result is the tested tree byte for byte**: the same git tree hash as 267308b4.

PR	Branch	Own commits	Files in its diff
#424	fix/2026-10-23-sec-consensus	11	37
#425	fix/2026-10-23-sec-p2p	3	15
#426	fix/2026-10-23-sec-storage	9	23
#427	fix/2026-10-23-sec-crypto	10	35
#428	fix/2026-10-23-sec-rpc	6	15
#429	fix/2026-10-23-sec-confighost	8	41
#430	fix/2026-10-23-sec-mempool	2	9
#431	fix/2026-10-23-sto-crash	2	9
#432	fix/2026-10-23-sto-dirsync	2	8
#433	fix/2026-10-23-sto-runcheck	2	2
#434	fix/2026-10-23-parsers	7	21
#435	fix/2026-10-23-getblocks	8	6

APPENDIX A: WHAT IS STILL OPEN

- **Not proved.** Everything listed OPEN or not covered in Chapters 9, 10 and 12: the script-number opcodes (tested only), the end-to-end retarget, the safegcd inverse, batch inversion, wNAF, the group law, and the ten parser harnesses that did not close within 8 GB. The assembly parsers were out of scope.
- **Not re-run.** The parser area's full `run.sh` was not re-run after its last mutant change (about an hour of CBMC). Its mutation checks were not re-run against the fixed parsers either: one mutant's pattern no longer matches the fixed code.
- **Not yet published.** The four formal-verification areas are on their own local branches.
- **Known, pre-existing, unfixed:** - a stale header-index entry can misplace the locator fork point until the serve loop's reorg check runs; - deep blocks are served as compact blocks; - the log falls back to fd 0 if its file cannot be opened; - Core's minimum-chainwork rule for `getheaders` is not implemented; - at boot, any mempool.dat error discards the whole file, where Core keeps what loaded; - the STO-1 reorg ordering item.
- **Residuals** are listed in the commit messages of each batch, for example the one-announcer-per-orphan simplification in MPL-5.

APPENDIX B: BEHAVIOUR CHANGES FOR OPERATORS

- **Config:** errors in bitcoin.conf are now fatal at boot. RPC credentials apply only to their chain's section, as in Core.
- **Tor:** the control port must offer SAFECOOKIE. The plain cookie is never sent.
- **Esplora:** no wildcard CORS header, batches are bounded, and the listener has a fixed thread pool and total deadlines.
- **Relay:** unknown-witness-version spends are non-standard (not relayed or mined) and stay consensus-valid.
- **Reorgs:** there is no reorg depth cap. A heavier valid chain, or an `invalidateblock`, deeper than 100 blocks is followed, as in Core.
- **Boot:** a UTXO store that exists but will not reload, or whose manifest names a missing run, halts the boot with the tip held. It no longer starts on a partial set.
- **Sync cost:** the UTXO WAL buffer is 32 MB, and the store syncs block files, index and undo before each WAL drain: about 2 extra syncs per block at the tip.
- **Wallet:** opening a legacy v2 wallet file logs a SECURITY WARNING to treat it and every copy as plaintext and to move the funds.